Aran: Novel Data-Flow Firewall for Indian Banks

Oct 15, 2026 · @Inbathamizhan S

1. Problem analysis

The real problem is not "find PII with regex". It is that a bank cannot prove, at any moment, where each customer's identifiers are, who can read them in clear, and which copy leaked. Detection, masking, tracking and erasure are four views of that one missing guarantee.

What leaks, and where. Sensitive values leave their system of record through five channels: API payloads (request and response bodies, query strings, headers), application and debug logs, analytics and data-lake exports, support tools and shared reports, and now prompts sent to AI assistants and LLM APIs. Logs are the worst channel because they are written by every service, retained long, replicated to log platforms and read by many engineers.

The data types and their real structure. The structure of each identifier decides how well it can be detected. Three carry a mathematical check digit; OTPs carry none.

Data type	Real format	Built-in check	Detection difficulty
Aadhaar (UIDAI)	12 digits, first digit 2 to 9, often printed 4-4-4	Verhoeff check digit (dihedral group D5)	Low with the checksum, high without it
Payment card number	13 to 19 digits, issuer prefix (RuPay 60, 65, 81, 82, 508; Visa 4; Mastercard 51 to 55 and 2221 to 2720)	Luhn mod-10	Low
PAN (Income Tax)	5 letters, 4 digits, 1 letter; 4th letter = holder type (P person, C company, H HUF, F firm, T trust); 5th letter = surname or entity initial	A check letter whose algorithm is not public	Medium (rigid pattern, no public checksum)

Data type	Real format	Built-in check	Detection difficulty
UPI ID (VPA)	handle@psp, for example name@oksbi or 98xxxxxx@ybl	None; PSP handle list is finite and published by NPCI	Medium (looks like an email)
OTP	4 to 8 random digits, valid for minutes	None	Very high: indistinguishable from any short number
Supporting quasi-identifiers	10-digit mobile starting 6 to 9, IFSC (4 letters, 0, 6 characters), account number (9 to 18 digits), DOB, PIN code	None	High, but dangerous in combination

Threat model. Four actors cause leaks: (1) careless code that logs full objects, (2) insiders who copy exports or paste data into AI tools, (3) third-party processors and vendors who receive shared data, and (4) external attackers who exfiltrate stores or logs. The firewall must work without trusting application developers to call it correctly, so it sits on the paths data must cross: the API gateway, the log shipper, the export and file-share path, and an AI gateway.

Hard constraints from the problem statement.

- Protection must happen before storage or sharing, so it must run inline at line rate with bounded latency.
- Some systems still need the value (payments, KYC re-verification), so protection must be reversible for authorised callers and irreversible for everyone else.
- Tracking must survive copies the bank does not control, such as a vendor's database or a dump posted on Telegram.
- Erasure must reach every copy, including logs, backups and AI caches, while still respecting laws that force retention.

2. Baseline solution

The baseline is what most teams would build first: a detect-then-redact proxy. It works, and it is the yardstick the novel design must beat.

1. **Intercept** traffic at four choke points: an API gateway plugin (Envoy or Kong filter), a log-shipper processor (Fluent Bit or Vector), an export and file-share scanner, and an AI gateway in front of LLM APIs.

2. **Detect** with a layered recogniser: regular expressions for each format, checksum validation where one exists (Verhoeff, Luhn), context words ("aadhaar", "pan", "otp", "card") and an NER model for names and addresses. Microsoft Presidio already ships India recognisers for Aadhaar (with checksum), PAN, voter ID, passport, vehicle registration and GSTIN.
3. **Protect** by policy per field and destination: mask for display (XXXX XXXX 1234), tokenise with a vault for systems that must join on the value, and format-preserving encryption (NIST FF1) where schemas cannot change.
4. **Track** by writing every detection event (field, service, destination, time) to an audit store and drawing a lineage graph.
5. **Erase** on a DPDP request by looking up the customer's token and deleting rows in every store the lineage graph lists.

Where the baseline breaks.

- Pattern detectors on numbers fire on order IDs, timestamps and block IDs, so teams raise thresholds and miss real leaks. OTPs cannot be found by pattern at all.
- A token or FPE ciphertext looks exactly like a real number, so a scanner cannot tell a protected Aadhaar from a leaked one. Every downstream alert is ambiguous.
- Masking is treated as yes or no. Nobody measures how much a masked value still reveals, or what three masked fields reveal together.
- Lineage stops at the bank's boundary. When a dump appears on Telegram, the audit log cannot say which of 40 vendors leaked it.
- Row deletion cannot reach immutable logs, backups, vendor copies or LLM context caches, so DPDP erasure is a best effort, not a proof.

3. Literature survey

The survey covers the rules a bank must meet, real leaks that show the failure modes, and six research lines the design draws on. Each line solves one piece; none joins detection, protection, attribution and erasure under one guarantee.

3.1 Rules the firewall must satisfy

Rule	What it requires	What it means for the design
DPDP Act 2023, Schedule	Penalty up to Rs 250 crore for failing reasonable security safeguards (s. 8(5)); up to Rs 200 crore for failing breach notice (s. 8(6))	Masking and encryption are the named safeguards; a leak without them is the costliest violation

Rule	What it requires	What it means for the design
DPDP Rules 2025, notified 13 Nov 2025	Core duties apply from 13 May 2027; detailed breach report within 72 hours; logs kept at least 1 year	The firewall must produce a 72-hour forensic answer, and erasure must coexist with mandatory log retention
CERT-In Directions, 28 Apr 2022	Report listed incidents within 6 hours; keep ICT logs 180 days in India	Logs are kept by law, so they must be safe to keep: protect before write, not after
UIDAI Aadhaar Data Vault, Circular 8 of 2025	Aadhaar stored only in a vault with HSM and AES-256; business processes use reference keys	Tokens are mandatory for Aadhaar; the design extends the reference key idea to every sink
PCI DSS and 8-digit BINs	First 6 and last 4 is the format every brand accepts; FAQ 1091 lists brand maximums	Masking formats are set by brands, so the residual risk they leave must be measured, not assumed
NIST SP 800-38G Rev. 1, 2nd draft (Feb 2025)	FF3 removed; FF1 kept with a larger minimum domain	Use FF1 only, and never on tiny domains such as 4 to 6 digit fields

3.2 Real leaks and what each one teaches

Incident	What leaked and how	Lesson
Indian debit card breach, 2016	About 3.2 million cards (SBI, HDFC, ICICI, Yes, Axis) via malware at Hitachi Payment Services	Leaks happen at a processor, outside the bank; attribution needs evidence that survives the boundary
SBI Quick server, 2019	Unprotected server exposed balances and recent transactions from SMS service logs	Message and log stores leak, not only core databases
Juspay, 2020 (disclosed 2021)	35 million records of masked card data; an old AWS key was abused; found when sold on the dark web	Masked data still sells; the breach surfaced months later as an external dump

Incident	What leaked and how	Lesson
Star Health, 2024	Attacker claimed 31 million customers' records, sold through Telegram chatbots	The first evidence is often a public dump, and the first question is which system it came from
Meta passwords in logs, fined 2024	Hundreds of millions of passwords logged in plain text; EUR 91 million fine	Logs are a top leak channel even at the best-run firms
Samsung and ChatGPT, 2023	Three incidents of staff pasting source code and meeting notes into ChatGPT	AI tools are a new egress path that classic DLP does not see

Cost context: IBM put India's average breach at Rs 22 crore in 2025, with shadow AI among the top three cost drivers (IBM India). The 2026 edition reports Rs 25.5 crore, and Rs 40.9 crore for financial services, the highest sector (Deccan Herald).

3.3 Research lines

Line	Key works	What it gives	What it leaves open
PII detection in text	Microsoft Presidio (India recognisers incl. Aadhaar checksum); Pilán et al. 2022, Text Anonymization Benchmark; Lison et al. 2021, ACL survey	Regex, checksum and NER recognisers with context words	Scores are not calibrated probabilities; no OTP detection; numeric false alarms in logs
PII leakage from LLMs	Carlini et al. 2021, USENIX Security; Lukas et al. 2023, IEEE S&P	Shows models memorise and emit PII; scrubbing reduces but does not remove it	Assumes scrubbing is the defence; no reversible tokens for prompts
Taint and information-flow tracking	Enck et al. 2010, TaintDroid (OSDI); Arzt et al. 2014, FlowDroid (PLDI); Pasquier et al. 2017, CamFlow provenance	Tracks data inside one runtime or host precisely	Needs instrumentation of every service; stops at the network or vendor boundary

Line	Key works	What it gives	What it leaves open
Format-preserving encryption	Bellare et al. 2009, FPE; NIST SP 800-38G (FF1); Durak and Vaudenay 2017 (FF3 attack); Beyne 2021 (FF3-1 linear attack)	Encryption that keeps the 12 or 16 digit shape	Ciphertext is indistinguishable from real numbers, so scanners cannot tell safe from unsafe
Secure deletion and crypto-shredding	Boneh and Lipton 1996, revocable backups; Perlman 2005, Ephemerizer; Geambasu et al. 2009, Vanish; Green and Miers 2015, puncturable encryption	Delete a key, lose all its ciphertexts, including backups	Breaks for deterministic tokens shared by many records; ignores legal-hold retention
Machine unlearning	Cao and Yang 2015 (IEEE S&P); Bourtoule et al. 2021, SISA	Removing a person from a trained model	Expensive; better to keep identifiers out of models in the first place
Leak attribution and tracing	Chor, Fiat and Naor 1994, traitor tracing; Boneh and Shaw 1998 and Tardos 2008, fingerprinting codes; Juels and Rivest 2013, honeywords	A leaked copy reveals which recipient leaked it	Built for content distribution, not for per-field identifiers in banking data flows
Source localisation on graphs	Shah and Zaman 2011, rumour centrality (IEEE Trans. IT); Pinto, Thiran and Vetterli 2012 (Physical Review Letters)	Maximum-likelihood source of a spread from a few observations	Never applied to data lineage graphs of a bank
Quantitative information flow	Smith 2009, min-entropy leakage (FoSSaCS); Sweeney 2000, 87% of the US unique by ZIP, sex and birth date	Measures leakage in bits, and shows small fields re-identify	Not used to set masking policy, and no budget across many sinks

Line	Key works	What it gives	What it leaves open
Check-digit theory	Luhn 1960 (US patent 2,950,048); Verhoeff 1969; Gumm 1985 (IEEE Trans. IT)	Check digits catch all single-digit errors; Verhoeff also catches all adjacent swaps	Only used for input validation, never as a security invariant

4. Research gaps

Six gaps remain open, and the first three are measurable today on real logs. Our test on 32,000 real production log lines from 16 systems (Loghub, section 7) shows the size of each.

Gap	Evidence	Research problem
G1. Number detectors drown in false alarms	A plain OTP pattern fired 67,481 times on 32,000 lines; a card pattern fired 6,086 times; context words matched "otp" inside "hotplug"	Detection must be a calibrated decision with a provable false alarm rate, not a regex score
G2. Protected and leaked values look the same	FF1 and vault tokens keep the 12 or 16 digit shape and a valid-looking form	Make protection itself detectable: a token must be provably distinguishable from a real identifier
G3. Masking is judged yes or no	First 8 and last 4 of a 16-digit card leaves only 1,000 candidates (about 10 bits); first 4 and last 4 of Aadhaar also leaves 1,000	Measure what a mask leaks in bits and cap what any sink, or any group of sinks, learns in total
G4. Leaks are attributed by access logs, not by the leaked data	Juspay and Star Health surfaced as outside dumps months after access	The leaked bytes themselves must name the system that leaked them
G5. Erasure cannot reach logs, backups, vendors and AI caches, and clashes with legal retention	DPDP and CERT-In both force log retention; RBI KYC rules force record retention	Erasure that reaches every copy in one operation, scoped by purpose, with a proof

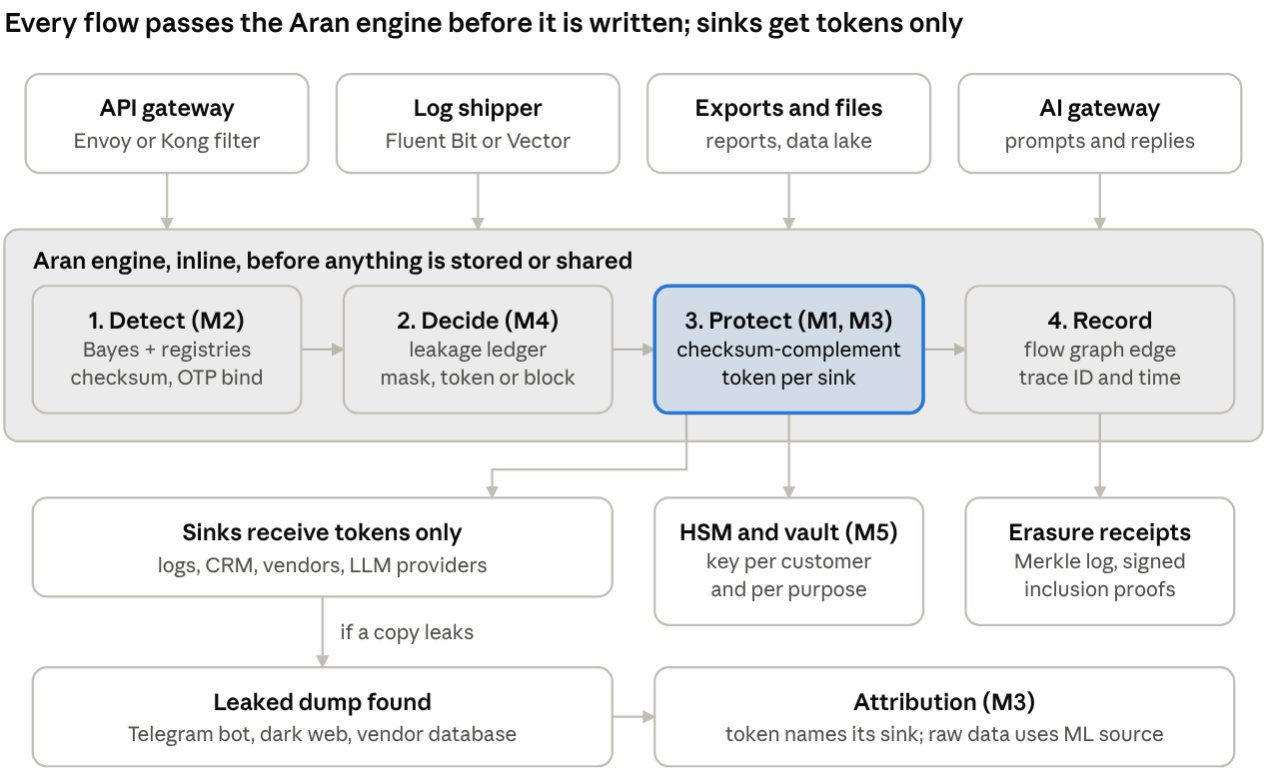
Gap	Evidence	Research problem
G6. OTPs have no structure at all	Any 4 to 8 digit number looks like an OTP	Detect a secret by what it is (an issued secret), not by how it looks

5. Novel solution: Aran, an algebraic data-flow firewall

The core idea: turn the check digit that every Aadhaar and card number already carries into a security invariant. Real identifiers always satisfy their checksum; Aran's tokens are built to always fail it. That one algebraic fact makes protected data provably distinguishable from leaked data, and the rest of the design (attribution, leakage budgets, erasure) is built on top of it.

Aran has five mechanisms, each derived from a named theory.

Mechanism	Theory it rests on	Gap closed
M1. Checksum-complement tokens	Group theory (dihedral group D5 for Verhoeff, modular arithmetic for Luhn) and pseudorandom permutations (FF1)	G2
M2. Evidence-fusion detector with issued-secret registries	Bayesian decision theory and the Neyman-Pearson lemma; Bloom filter error bounds	G1, G6
M3. Sink-tweaked tokens for leak attribution	Pseudorandom permutation families, birthday bounds; maximum-likelihood source estimation	G4
M4. Leakage ledger	Quantitative information flow (min-entropy)	G3
M5. Purpose-bound crypto-shredding with erasure receipts	Perfect secrecy (Shannon) and Merkle transparency logs	G5



Aran architecture · 4 choke points, 4 inline steps, forensic path

Data enters at four choke points, passes the four inline steps, and leaves only as tokens; the HSM holds the keys that erasure destroys, and any leaked copy routes back to its sink.

M1. Checksum-complement tokens

An Aadhaar number is 11 payload digits x plus a Verhoeff check digit c(x). Write V(s) = 1 when a string passes Verhoeff. Aran's token for sink s keeps the shape but forces a wrong check digit, using a keyed offset δ between 1 and 9:

```
T_s(x) = E_{K}^{\,s}(x) \,\Vert\, \bigl( c(E_{K}^{\,s}(x)) + \delta_K(s)
\bigr) \bmod 10, \qquad \delta_K(s) \in \{1,\dots,9\}
```

Here E is FF1 with the sink name as tweak (vaultless mode), or a uniform random 11-digit string stored in the vault (vault mode). Cards use the same construction with Luhn.

Theorem 1 (perfect separability). For every real Aadhaar a, V(a) = 1. For every token t, V(t) = 0.

Proof sketch. Verhoeff detects every single-digit error, so for a fixed 11-digit body exactly one final digit passes. The token's final digit differs from that one by δ, which is never 0 mod 10. Hence no token passes. ∎

Consequence: a scanner anywhere (log platform, data lake, a dump found online) can classify every 12-digit hit with zero ambiguity. A checksum-valid hit is raw data that escaped protection, and a checksum-invalid hit in a token field is protected. Detokenisation is lossless because the true check digit is recomputed from the decrypted body. The experiment confirms it: 20,000 tokens injected into real logs, 0 flagged; 20,000 raw values, all flagged.

M2. Evidence-fusion detector with issued-secret registries

Each candidate string gets a posterior probability that it is a real identifier, not a regex score. Independent evidence multiplies as likelihood ratios:

```
\frac{P(\text{real}\mid e)}{P(\text{noise}\mid e)} = \frac{\pi}{1-
\pi}\prod_{i} \Lambda_i,\qquad \Lambda_{\text{checksum}} = \frac{P(V{=}1\mid
\text{real})}{P(V{=}1\mid \text{noise})} = \frac{1}{0.1} = 10
```

The Neyman-Pearson lemma says thresholding this ratio is the most powerful test at any chosen false alarm rate, so the bank sets the false alarm rate it can afford and the threshold follows.

The strongest evidence comes from registries the bank already owns:

- **Own-customer registry.** Keyed hashes HMAC(K, id) of every Aadhaar, PAN, card and account number the bank holds go into a Bloom filter. A hit gives a likelihood ratio of 1/ε, where the Bloom false-positive rate is ε = (1 − e^(−kn/m))^k. With 10^8 identifiers, m = 2.9 × 10^9 bits (about 360 MB) and k = 20 hashes, ε is about 10^−6. The filter reveals nothing without K, which stays in the HSM.
- **Issued-OTP registry, bound to the trace.** OTPs have no structure, so Aran detects them by what they are. The OTP service registers HMAC(K, otp ‖ trace_id) at issue time with a time-to-live. A 4 to 8 digit string in a log line is an OTP leak only if its HMAC with that line's W3C trace ID is live. Without binding, a random 6-digit number would match one of L live OTPs with probability L/10^6, which is 7% at 70,000 live OTPs. With binding it is 10^−6 per candidate.

M3. Sink-tweaked tokens for leak attribution

Every destination (log platform, CRM, each vendor, the LLM gateway) gets its own tweak, so the same customer has a different token in every sink. A token found anywhere names its sink.

Theorem 2 (attribution). With a pseudorandom permutation per sink over a domain of size D = 10^11, N customers and S sinks, the chance that a leaked token is wrongly attributed is at most

```
P(\text{misattribution}) \le \frac{(S-1)\,N}{D} \quad\text{per token,}\qquad
P(\text{wrong sink for a dump of } k \text{ tokens}) \le \left(\frac{(S-1)N}
{D}\right)^{k}
```

For a bank with N = 5 × 10^7 customers and S = 40 sinks, one token is mis-attributed with probability at most 0.02, and a dump of 5 tokens at most 3 × 10^−9. In vault mode the vault rejects duplicates, so the bound is 0. The experiment traced every dump of 1, 2, 5 and 50 tokens to the right sink among 40 (200 trials each, 0 token collisions among 800,000 tokens).

When a dump holds raw values (leaked before Aran or from a system of record), Aran falls back to maximum-likelihood source estimation, in the spirit of rumour-source detection. Each system v holds a set H_v of keyed-hash identifiers, kept as MinHash sketches. For a dump L the most likely source is

```
\hat v = \arg\max_v \; \pi_v \prod_{x\in L} \Bigl[(1-\eta)\,\tfrac{\mathbf
1[x\in H_v]}{|H_v|} + \eta\,\tfrac{1}{|U|}\Bigr]
```

where η allows for noise. The formula prefers the smallest system whose holdings contain the whole dump, which is exactly the system a forensic analyst would suspect first. The newest record in the dump also bounds the leak time from below.

M4. Leakage ledger

A mask is judged by how many candidates it leaves, measured as min-entropy $H_\infty(X \mid \text{view})$ = log2 of the number of identifiers consistent with what a sink has seen. Aran counts this exactly with a dynamic programme over the checksum group, in O(length × 10 × |G|) steps.

Policy. For every trust domain D (sinks that can join data, such as all systems of one vendor), and every secret X of a customer:

```
H_\infty\Bigl(X \,\Bigm|\, \textstyle\bigcup_{s\in D}
\mathrm{view}_s(X)\Bigr) \;\ge\; \tau_{\text{type}}
```

If a new flow would push a domain below the threshold τ (for example 20 bits), Aran shows fewer digits or sends a token. This catches the case a per-field rule misses: one vendor system sees the first 8 digits of a card, another sees the last 4, and together they leave about 1,000 candidates.

M5. Purpose-bound crypto-shredding with erasure receipts

DPDP erasure and legal retention conflict, so Aran scopes keys by purpose. The HSM derives one key K(c, p) per customer c and purpose p (servicing, marketing, fraud, KYC legal hold). The vault stores Enc(K(c, p), x), and every sink sees only vault-mode tokens that were drawn at random.

Theorem 3 (erasure by key destruction). After K(c, p) and the vault index entries for c are destroyed, every copy of c′s token, in any log, backup, vendor database or AI cache, is statistically independent of c′s identifier.

Proof sketch. A vault-mode token is sampled uniformly and independently of x; the only link to x was the vault entry, which is now ciphertext under a destroyed key. With the link gone, the token carries zero bits about x (Shannon perfect secrecy). ∎

Two practical rules complete it. KYC keys carry a time-lock and are destroyed automatically when the legal retention ends, so erasure requests for that purpose are honoured on time without breaking the law. HSM backups rotate every Δ days, so erasure is complete by request time + Δ, and that bound is stated in the receipt. Every key destruction is appended to a Merkle transparency log (the structure behind Certificate Transparency, RFC 6962); the customer receives a signed inclusion proof as a verifiable erasure receipt.

The AI tools channel

The AI gateway applies M1 to M4 to prompts: identifiers become typed tokens such as ⟨PAN_7⟩ or checksum-complement numbers, the model works on tokens, and the reply is detokenised only for the authorised user. The model provider never receives a raw identifier, and anything it memorises is a token that M3 can trace back to the AI gateway.

6. Critical review

The design survives review, but eleven attacks forced changes; the fixes below are now part of Aran. The two most serious were insiders who detokenise before leaking, and typo′d identifiers that fail the checksum and look like tokens.

Attack or weakness	Severity	Fix now in the design
An insider with detokenise rights leaks raw values, so no token names the sink	High	Every detokenise call is logged per caller. A raw dump is intersected with each caller′s detokenise log; the caller whose log contains the dump is the ML source under M3

Attack or weakness	Severity	Fix now in the design
A real Aadhaar with one typo fails Verhoeff and is treated as a token	High	Verhoeff detects every single-digit error and adjacent swap, so for a failing string Aran tries the 108 single-digit edits and 11 swaps against the own-customer registry; a hit is a leak
Vaultless FF1 tokens cannot be crypto-shredded, because one sink key decrypts every customer	High	Data in DPDP scope uses vault-mode random tokens only (Theorem 3). Vaultless FF1 is limited to short-lived flows where erasure is not required
A Bloom filter cannot delete, so erased customers stay in the registry	Medium	Use a cuckoo filter (Fan et al. 2014), which supports deletion at a similar false-positive rate
The registry is a list of every customer's identifiers	Medium	Entries are HMAC values under an HSM key; without the key, testing one Aadhaar costs an HSM call, which is rate-limited and logged
Identifiers hidden by encoding (URL, base64, JSON escapes, split fields)	Medium	A normaliser decodes common encodings and joins adjacent numeric fields before detection
Naive Bayes multiplies correlated evidence and becomes overconfident	Medium	Calibrate posteriors with isotonic regression on labelled bank data; report calibration error with every model update
Min-entropy assumes identifiers are uniform; real issuance is not	Medium	Treat ledger bits as an upper bound and set τ with margin; refine with guessing entropy when issuance statistics are available
OTP binding needs a trace ID in the log line	Medium	Legacy systems without trace IDs fall back to context rules with word boundaries, which removes the "hotplug" false alarm seen in the experiment
FF1 on tiny fields (OTP, last 4, CVV) is weak	Low	Never FPE a domain under 10^6, per the NIST Rev. 1 draft; tiny fields are vaulted or dropped

Attack or weakness	Severity	Fix now in the design
Crypto-shredding is not named in DPDP as erasure	Legal	The argument is that tokens become anonymous data once the link is destroyed (Theorem 3); this needs sign-off from the bank's counsel and, ideally, guidance from the Data Protection Board

What is genuinely new, stated honestly. Some payment tokenisation products already offer tokens that fail Luhn. The new parts are: extending this to Aadhaar's Verhoeff code with a keyed offset; using the checksum as a proven detection invariant across the whole data estate (Theorem 1); combining it with per-sink tweaks so a leaked token names its source (Theorem 2); a min-entropy ledger that caps what groups of sinks learn; and purpose-bound, time-locked crypto-shredding with verifiable receipts (Theorem 3). No surveyed paper or product joins these, and each piece rests on a stated theorem rather than a heuristic.

7. Evaluation, implementation plan and sources

7.1 Results on real data

All four experiments ran on real, public, unsanitised production logs: the Loghub 2k samples from 16 systems (HDFS, Hadoop, Spark, OpenStack, Linux, Windows, Mac, Android, OpenSSH, Apache and others), 32,000 lines in total (Loghub, Zhu et al., ISSRE 2023). These logs contain no Indian identifiers, so every hit is a false alarm. No real person's identifier was used; values injected in E2 and E3 are generated with valid checksums. Code and raw numbers are on the research branch; FF1 passes the NIST SP 800-38G sample vectors.

Experiment	Baseline result	Aran result
E1. OTP detection on 32,000 real lines	Pattern: 67,481 false alarms	Context rule: 6, all from substrings such as "hotplug"; trace-bound registry: about 10^−6 per candidate by construction

Experiment	Baseline result	Aran result
E1. Card detection	Pattern: 6,086 false alarms	Luhn: 518; Luhn plus issuer prefix: 81, all HDFS block IDs that pass Luhn by chance; own-customer registry removes these (expected 81 × 10^−6)
E1. Aadhaar detection	Pattern: 366 false alarms	Verhoeff: 4 (fan-speed readings and UUID fragments), a 91-fold cut
E2. Token separability, 20,000 injections each	FF1 tokens pass as real numbers	20,000 of 20,000 raw values flagged; 0 of 20,000 checksum-complement tokens flagged
E3. Leak attribution, 20,000 customers, 40 sinks	Access logs only	0 collisions among 800,000 tokens; correct sink for 100% of dumps of 1, 2, 5 and 50 tokens (200 trials each)

E4 measured what common masks leave, by exact count:

Mask	Candidates left	Bits left
Card 16-digit, last 4 shown	100,000,000,000	36.5
Aadhaar, last 4 shown (UIDAI masked Aadhaar)	8,000,000	22.9
Mobile, last 4 shown	400,000	18.6
Card 16-digit, first 6 and last 4 shown	100,000	16.6
Card 16-digit, first 8 and last 4 shown	1,000	10.0
Aadhaar, first 4 and last 4 shown	1,000	10.0

Two masks that look reasonable leave only 1,000 candidates, which a script tries in seconds; this is why M4 budgets bits instead of trusting formats.

7.2 Implementation plan

1. **Core library in Rust:** Verhoeff and Luhn, FF1, checksum-complement tokens, cuckoo-filter registries, the min-entropy counter. Port the Python prototype and its tests.
2. **Choke-point adapters:** an Envoy filter for APIs, a Vector transform for logs, an export scanner, and an OpenAI-compatible AI gateway. Each calls the same core.

3. **Vault and keys:** vault-mode tokens, per-customer and per-purpose keys in an HSM (or a cloud KMS for the prototype), time-locks for KYC retention, Merkle erasure log with signed receipts.
4. **Flow graph and forensics:** record each protect event as a graph edge (source, sink, trace ID, time); build the attribution service for token dumps, raw dumps and the detokenise-log join.
5. **Ledger and policy:** min-entropy per secret per trust domain, with the threshold τ set by data type and sink class.
6. **Evaluation at scale:** full Loghub sets (tens of millions of lines), a labelled set of Indian banking log templates with injected generated identifiers, latency per event, and calibration error of the detector.

7.3 Novel contributions

- Checksum-complement tokens with a proof of perfect separability (Theorem 1).
- Per-sink tokens whose leaked copies name their source, with an explicit bound (Theorem 2), plus a maximum-likelihood fallback for raw dumps.
- A trace-bound issued-secret registry that makes structureless OTPs detectable.
- A min-entropy leakage ledger that budgets what groups of sinks learn.
- Purpose-bound, time-locked crypto-shredding with verifiable erasure receipts that reconciles DPDP erasure with legal retention (Theorem 3).

7.4 Sources

Regulation and incidents are linked inline in sections 3.1 and 3.2. Research works:

- Arzt et al., FlowDroid, PLDI 2014. Bellare, Ristenpart, Rogaway and Stegers, Format-Preserving Encryption, SAC 2009. Beyne, Linear cryptanalysis of FF3-1 and FEA, CRYPTO 2021.
- Bloom, Space/time trade-offs in hash coding, CACM 1970. Boneh and Lipton, A revocable backup system, USENIX Security 1996. Boneh and Shaw, Collusion-secure fingerprinting, IEEE Trans. IT 1998.
- Bourtoule et al., Machine Unlearning, IEEE S&P 2021. Cao and Yang, Towards making systems forget, IEEE S&P 2015. Carlini et al., Extracting training data from large language models, USENIX Security 2021.
- Chor, Fiat and Naor, Tracing traitors, CRYPTO 1994. Durak and Vaudenay, Breaking the FF3 format-preserving encryption standard, CRYPTO 2017. Enck et al., TaintDroid, OSDI 2010.

- Fan, Andersen, Kaminsky and Mitzenmacher, Cuckoo filter, CoNEXT 2014. Geambasu et al., Vanish, USENIX Security 2009. Green and Miers, Forward secure asynchronous messaging from puncturable encryption, IEEE S&P 2015.
- Gumm, A new class of check-digit methods for arbitrary number systems, IEEE Trans. IT 1985. Juels and Rivest, Honeywords, ACM CCS 2013. Laurie, Langley and Kasper, Certificate Transparency, RFC 6962, 2013.
- Lison et al., Anonymisation models for text data, ACL 2021. Lukas et al., Analyzing leakage of PII in language models, IEEE S&P 2023. Luhn, US patent 2,950,048, 1960.
- NIST SP 800-38G, FF1 and FF3 (2016) and Rev. 1 second draft (2025). Pasquier et al., CamFlow, SoCC 2017. Perlman, The Ephemerizer, ACSAC 2005.
- Pilán et al., The Text Anonymization Benchmark, Computational Linguistics 2022. Pinto, Thiran and Vetterli, Locating the source of diffusion in large-scale networks, Physical Review Letters 2012. Shah and Zaman, Rumors in a network: who's the culprit?, IEEE Trans. IT 2011.
- Smith, On the foundations of quantitative information flow, FoSSaCS 2009. Sweeney, Simple demographics often identify people uniquely, CMU 2000. Tardos, Optimal probabilistic fingerprint codes, JACM 2008. Verhoeff, Error detecting decimal codes, 1969.
- Microsoft Presidio supported entities. Loghub, Zhu et al., ISSRE 2023.